Agent Code Explanation

How to explain every major agent and orchestrator file

High level idea

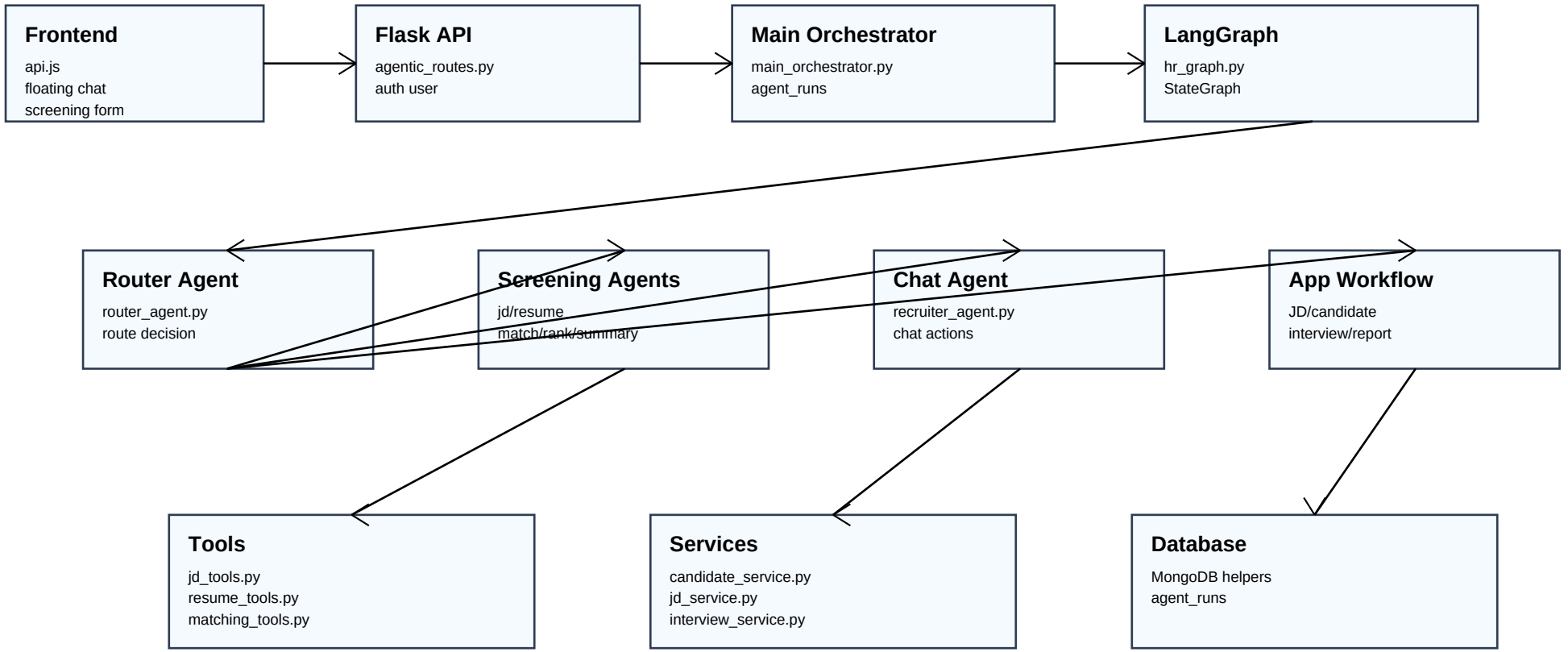
The project uses one Main HR Orchestrator and one LangGraph StateGraph. The graph starts at a Router Agent, then sends the request to a role-specific path such as screening, chat, JD, candidate, interview, dashboard, reports, or unsupported.

Code layers to explain

- Flask route layer: receives requests, files, and authenticated user context.
- Main orchestrator: creates run_id, stores agent_runs status, invokes the graph.
- LangGraph graph: defines nodes, conditional routing, and final response normalization.
- Agents: role-specific code for routing, JD loading, resume extraction, matching, ranking, summary, chat, and interview questions.
- Tools and workflows: reusable helpers for database, matching persistence, recruiter context, and application tasks.

Architecture Diagram

Code modules and runtime flow



Orchestrator Code

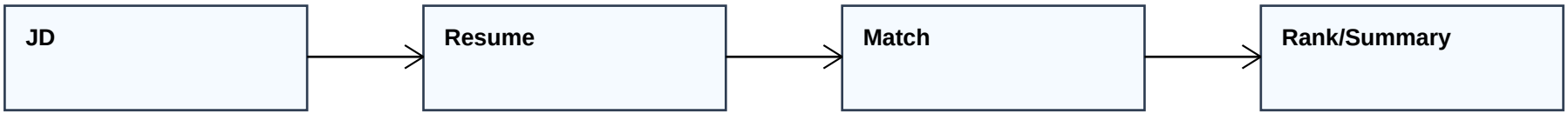
What to say about main_orchestrator.py and hr_graph.py

_task_type	Infers the task label used for run tracking from task_type, type, message, or jd_id.
_safe_input	Removes private fields and uploaded file objects before saving run input metadata.
run_main_orchestrator	Creates run_id, writes running status, invokes run_hr_graph, then writes completed or failed output.
HRGraphState	Typed dictionary that carries payload, route, contexts, errors, path result, and final response between nodes.
router_node	Runs route_request and stores route_decision plus route in graph state.
route_after_router	Conditional edge function that decides which branch runs after the router.
final_response_node	Converts each branch result into the stable API response shape.
build_hr_graph	Registers LangGraph nodes and edges, then compiles the graph once.

Screening Agents

What each screening function does

JD Agent	run_jd_agent loads the JD row and normalized JD JSON. build_jd_agent_prompt documents what the JD Agent would extract.
Resume Agent	run_resume_agent builds profiles from existing candidates and uploaded resumes. It records non-fatal extraction errors.
Matching Agent	run_matching_agent compares each profile with the JD and persists comparison rows through matching_tools.persist_match.
Ranking Agent	run_ranking_agent sorts results by match_score and splits selected_results from rejected_results.
Summary Agent	run_summary_agent produces the recruiter-facing summary, selected/rejected counts, top candidate, and errors.



Chat, Interview, and App Workflow Agents

How to explain non-screening branches

Router Agent	deterministic_route handles known task types. route_request uses deterministic routing first and LLM routing only when needed.
Recruiter Agent	run_recruiter_agent answers grounded recruiter questions. execute_recruiter_action runs supported chat actions with confirmation for side effects.
Interview Agent	run_interview_question_agent generates role-specific interview questions from candidate and JD context.
App Workflow	run_app_workflow dispatches JD, candidate, profile, dashboard, report, and interview tasks to existing services.
Agentic Routes	api_agentic_run normalizes request payloads, adds authenticated user context, and calls the Main HR Orchestrator.

UML Sequence Diagram

How one request moves through the code